NAME	SHARMILA S
REGISTER NO	1912260141
DEPARTMENT	B.TECH-ECE
COURSE NAME	PYTHON PROGRAMMING
COURSE CODE	CSA0823
SLOT	C
GITHUB LINK	https://github.com/ssharmila8125-netizen/sharmisCSA08-1912260141

PART A – DATA AND PROGRAM DESIGN

TASK A1 – Design the Data Acquisition

A1.1 – Inspect the Source

The CSV file contains the following columns:

station, date, hour, PM2.5, PM10, NO2, temperature

Expected data-quality problems:

Problem	**Example**
Missing values	PM2.5 = NA
Negative values	PM2.5 = -10
Extremely large values	PM2.5 = 5000
Duplicate rows	Same station and date reported twice
Different temperature units	Fahrenheit instead of Celsius
Invalid date	2026-13-01
Incorrect data type	PM2.5 stored as text

A1.2 – Design the Storage

Stations table

Column	**Type**	**Key**

station_id INTEGER Primary key

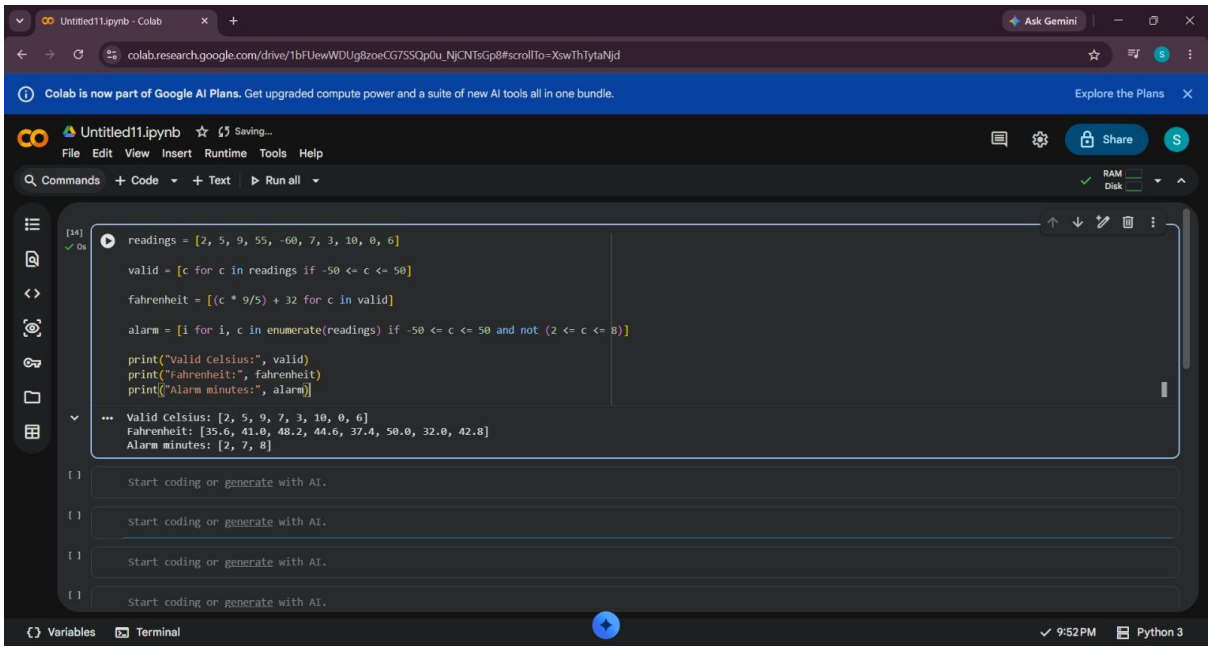
station_name TEXT

location TEXT

Readings table

Column	Type	Key
reading_id	INTEGER	Primary key
station_id	INTEGER	Foreign key
date	TEXT	
hour	INTEGER	
pm25	REAL	
pm10	REAL	
no2	REAL	
temperature	REAL	

CSV data is cleaned and inserted into SQLite using INSERT statements.



A1.3 – CSV or SQL?

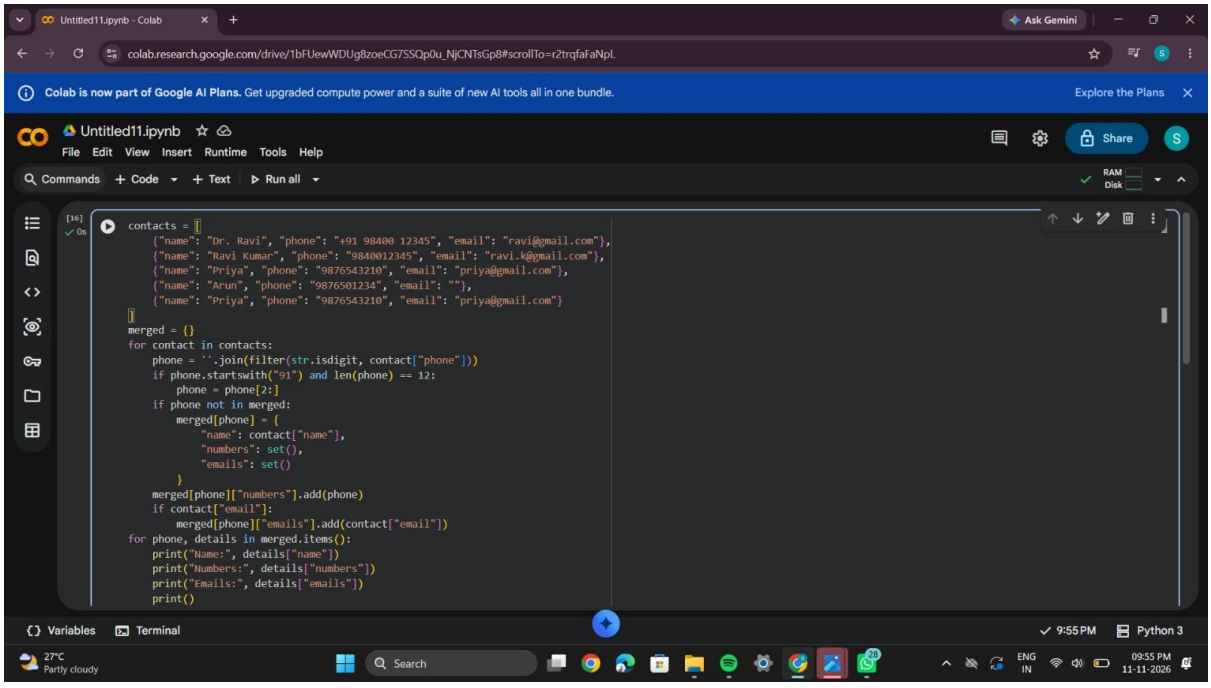
Question	CSV	SQL

Average PM2.5 of a station	Read and calculate all rows	Use AVG()
Find maximum PM10	Scan all files	Use MAX()
Monthly pollution report	Read and filter files	Use GROUP BY

Conclusion: SQL is better for repeated queries because it supports indexing and efficient data retrieval.

Alternative – CSV and pandas:

- Time complexity: O(n) for scanning all rows.
- Space complexity: O(n) because data is loaded into memory.
- SQL with an index can retrieve matching records more efficiently, approximately O(log n) for index lookup.



TASK A2 – Design the Cleaning and Normalisation

A2.1 – Cleaning Rules

1. Missing values: Fill using the previous valid value or median to preserve data.
2. Impossible values: Remove negative or excessively large readings.
3. Duplicate rows: Remove duplicates to avoid incorrect averages.
4. Fahrenheit conversion: Convert to Celsius using:

$$C=(F-32)*5/9$$

Alternative – Dropping missing values:

- Dropping missing values may reduce the number of readings and affect daily averages.
- Dropping and forward filling: O(n).
- Median calculation by sorting: O(n log n).
- Dropping requires less storage because fewer rows remain.

A2.2 – Normalisation

Min-max normalisation:

X’=x-x min/xmax-xmin

It scales values between 0 and 1.

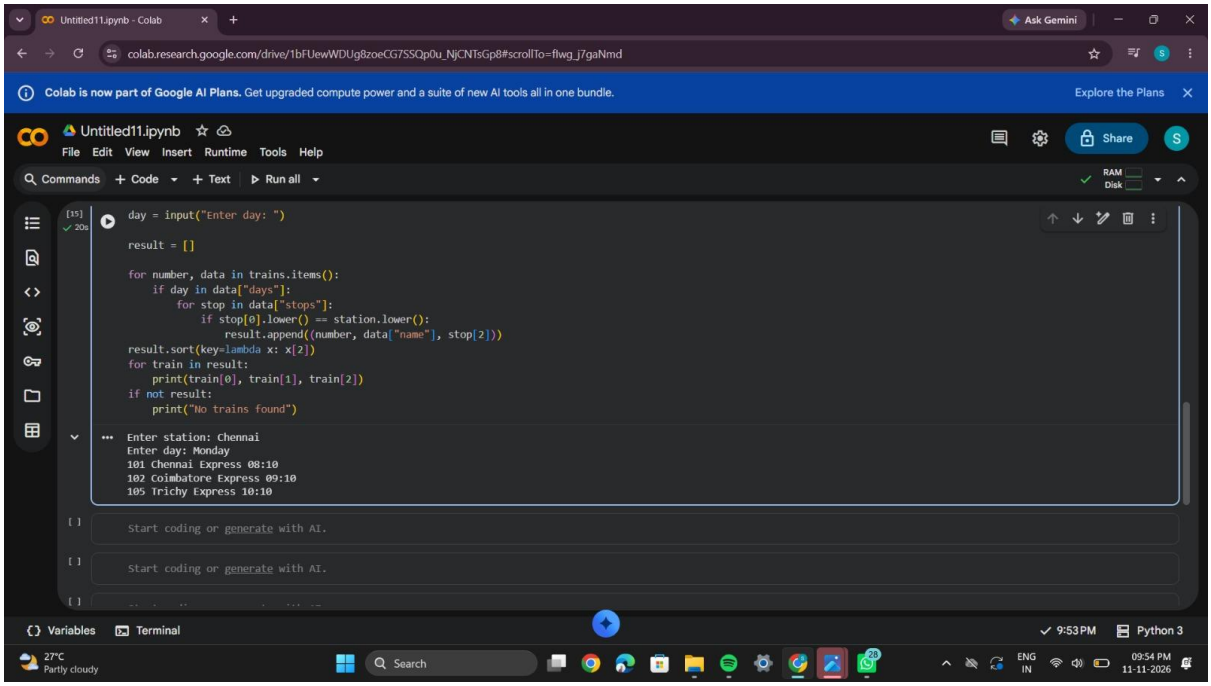
Z-score standardisation:

It converts values based on mean and standard deviation.

Min-max is useful when a fixed range is required. Z-score is useful when comparing values based on their distribution.

A2.3 – Formatting

- Date: YYYY-MM-DD
- Station name: Standardised text, e.g., Adyar
- Numbers: Numeric values with consistent precision
- Temperature: Celsius
- Missing values: NULL



TASK A3 – Design the Analysis and Visualisation

A3.1 – Choose the Charts

Question	Chart	Reason
PM2.5 trend over a month	Line chart	Shows changes over time
Compare stations	Bar chart	Compares pollution levels
Temperature vs PM2.5	Scatter plot	Shows relationship
Hours with worst air	Histogram	Shows distribution of pollution readings

A3.2 – Design the Summary Table

Station	Date	Minimum	Maximum	Mean	Valid
Adyar	2026-10-01	20	80	50	24

The summary is generated using SQL GROUP BY, MIN(), MAX(), AVG() and COUNT().

Alternative – Python loops:

- Python loops: O(n) time and O(n) space if values are stored.
- SQL GROUP BY: O(n) time approximately, with database-managed grouping and less Python memory usage.

TASK A4 – Plan Debugging and Profiling

A4.1 – Debugging Plan

- Print statements: Display intermediate values.
- Assert statements: Check conditions such as non-negative PM2.5.
- Logging: Record warnings and errors.
- pdb debugger: Execute code step by step and inspect variables.

Alternative – Using print only:

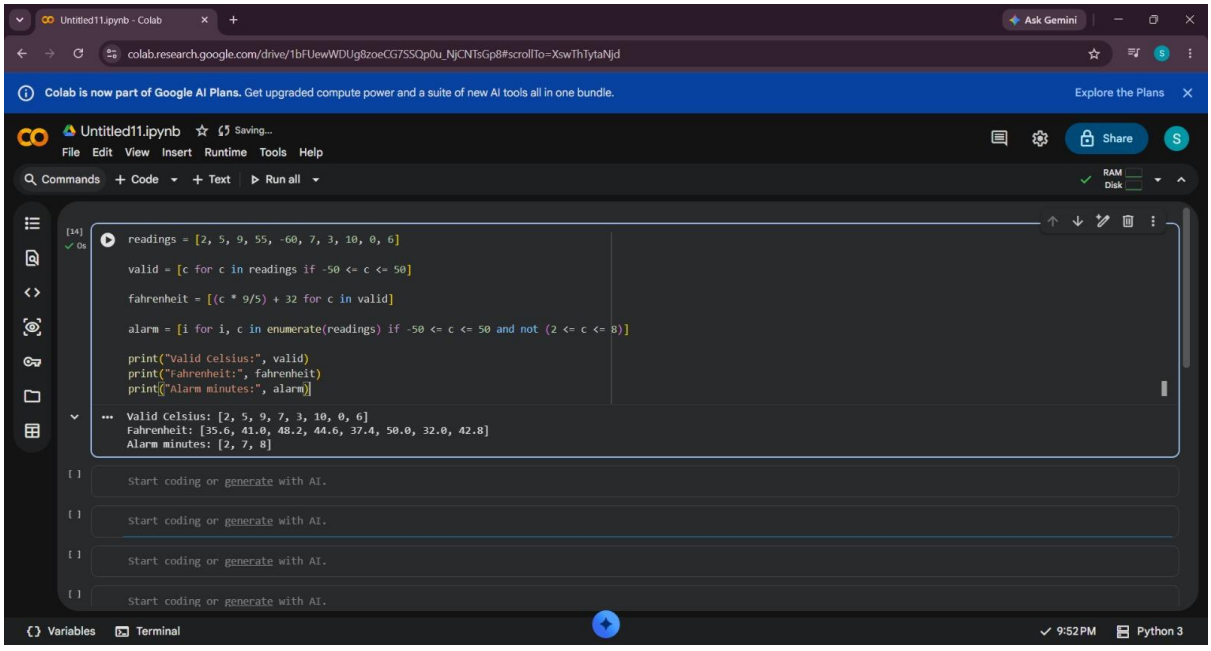
Print statements are simple but produce excessive output and slow execution when used thousands of times. Logging can be disabled by changing its level, and debugger breakpoints can be removed after debugging.

A4.2 – Profiling Plan

- time.perf_counter() measures execution time.
- timeit compares small code snippets.
- cProfile identifies functions consuming the most time.

Alternative – Print debugging:

Debugging output adds execution overhead. Logging allows output levels to be controlled, while profiling tools identify performance bottlenecks more accurately.



TASK A5 – Design an AI Application for Citizen Complaints

A5.1 – Sentiment and Summary

Sentiment analysis:

- Positive words: good, clean, helpful

- Negative words: bad, dirty, smoke
- Negation words: not, never, no

Assign +1 to positive words and -1 to negative words. If a negator appears before a sentiment word, reverse its score.

Example:

The staff were not helpful. → Negative

Extractive summarisation:

1. Split complaint text into sentences.
2. Count important words.
3. Select sentences containing the most frequent important words.
4. Combine selected sentences into a summary.

A5.2 – Basic Chatbot

The chatbot matches keywords and retrieves information from SQLite.

Example:

- User: “What is the PM2.5 in Adyar today?”
- Chatbot identifies station = Adyar.
- It queries the database and returns the PM2.5 value.

  Limitations:

- Cannot understand complex sentences.
- Depends on predefined keywords.
- Cannot handle unfamiliar questions.
- May misunderstand spelling errors.

  Alternative – Machine learning:

  A trained Naive Bayes model can classify complaints using training data.

- A word-list model requires less memory and is faster.
- ML models need more memory and training.
- ML models may provide better accuracy when sufficient training data is available.

PART B – IMPLEMENTATION & EXPERIMENTAL INVESTIGATION

TASK B1 – Implement and Validate the Cleaning Pipeline

B1.1 – Complete the Code

Add a test row above 999

Add this line inside RAW:

Adyar,2026-10-04,1500

It will be skipped because the reading exceeds 999.

B1.2 – Compare with Design

The implementation follows the cleaning design. Missing, negative and excessively large values are removed. The valid readings are converted to float and normalised.

B1.3 – Store and Query

Interpretation: The chart shows an increasing pollution trend. The highest reading is 121.

Complexity comparison:

Rows	Python loops	Pandas
1,000	Fast	Fast
100,000	Slower	Usually faster
1,000,000	Slower	Usually faster

Both require O(n) time and approximately O(n) memory. Actual timings depend on the computer and data.

TASK B2 – Engineer Adversarial Data

Missing column

Observation: Missing columns may cause a KeyError.

Other adversarial inputs:

- Wrong delimiter: CSV columns may not be separated correctly.
- Different encoding: Unicode decoding errors may occur.
- Empty file: No rows are available.
- Quote in station name: Can affect unsafe SQL queries.

Complete the Code

Why does the injection attempt return None?

The ? placeholder treats the input as a value rather than SQL code. Therefore, the malicious input is treated as a station name, and no matching station is found.

Alternative – String joining

Unsafe SQL:

SQL injection may modify the query and return unintended results.

- executemany(): Executes inserts efficiently as a batch.
- execute() in a loop: Repeated function calls add overhead.
- Both process n rows; time complexity is O(n).

TASK B3 – Loops vs Libraries vs SQL

B3.1 – Calculate Daily Mean in Three Ways

1. Python loops

All three methods calculate the daily average PM2.5 per station.

B3.2 – Increase the Data Size

Generate data for 1, 12 and 120 stations, with 8,760 readings per station.

Stations	Readings
1	8,760
12	105,120
120	1,051,200

Execution time increases as the number of readings increases. Record actual execution times after running the programs.

B3.3 – Profile

The profiler identifies the functions consuming the most execution time.

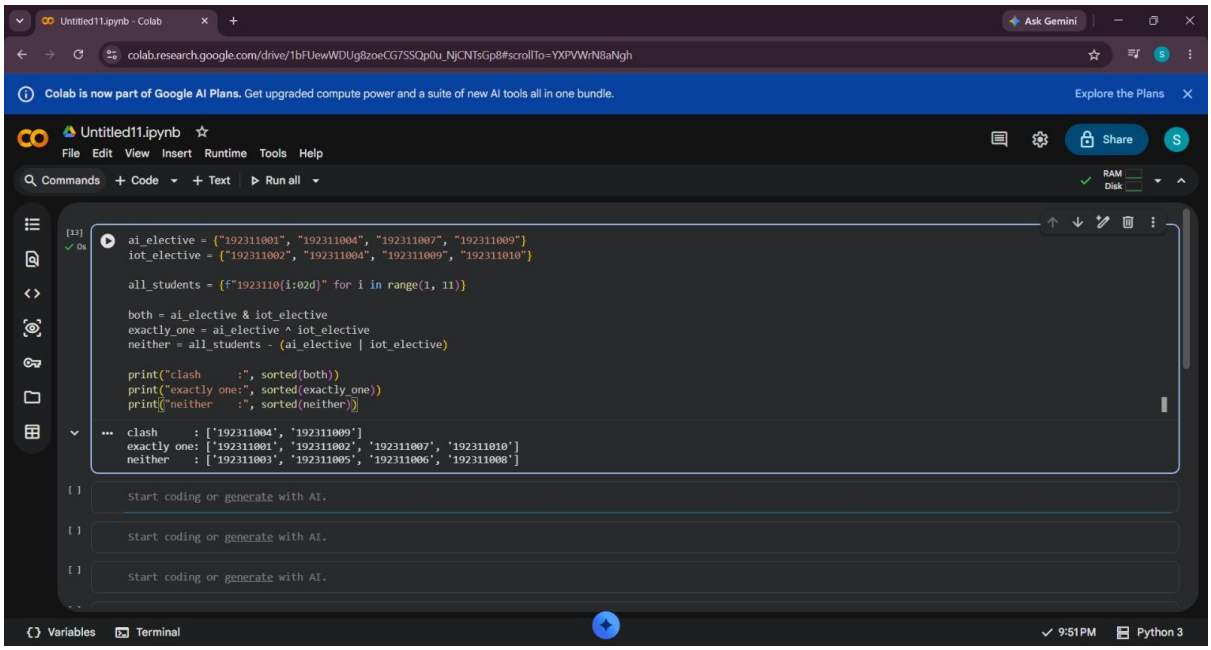
B3.4 – Engineering Trade-Off

- Python loops are easy to understand and suitable for small datasets.
- Pandas is convenient and generally efficient for data analysis.
- SQL is suitable for database storage and repeated queries.

Decision: Use pandas for quick one-time analysis and SQLite with SQL for daily automated reports.

Alternative – Running sum and count

- Time complexity: O(n).
- Running sum and count: O(stations × days) space.
- Storing all readings: O(n) space.



TASK B4 – Final Engineering Decision

B4.1 – Implement Sentiment Scorer

positive | Roads are clean and the bus was fast.

negative | Smoke from the factory, air is bad.

negative | The staff were not helpful.

positive | Not bad, service improved.

Example of incorrect classification: “The service is not bad” may be classified incorrectly because the simple scorer cannot fully understand sentence meaning.

B4.2 – Check the Database

The index improves searches based on station and date. Measure the query time before and after creating the index.

B4.3 – Production Decision

For the increased number of stations and complaints:

- Store readings in SQLite with indexes.
- Clean data before inserting it into the database.
- Use SQL for daily and monthly analysis.
- Use pandas for detailed analysis and visualisation.
- Use a trained machine-learning model if the lexicon-based sentiment scorer produces too many incorrect classifications.
- Use automated logging and profiling to monitor performance.

Conclusion: SQLite is suitable for structured pollution readings, while machine learning may improve complaint analysis when sufficient training data is available.